SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA37**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

CSA8714 - SOFTWARE TESTING

Date: 15/10/26.

Name: Dharani E

CO1-AT2

Reg No: 192321120

1) Testing as an Environment, engineer activity:

1, 6, 8, 10, 11, 21, 23 24, 29, 30.

Software testing as an engineering activity means following a systematic & planned approach rather than random Testing.

Process of Software Testing:

1. Test planning: Define Objectives, Scope, resources, schedule to conduct that test
2. Test design: Prepare test cases based on Requirement using test case principles
3. Test execution: Execute test cases using suitable input
4. Defect reporting: Record & track identify defects.

6) Importance of early testing involvement:

A well defined testing Process ensures that software is tested automatically the software testing should be well defined which will the classification of defects easily systematically & reliably

* Requirement analysis identify testing needs
* testing planning defines the test strategy
* test case design specifies inputs and outputs.

8) Boundary Value Analysis:

It's a Black Box testing technique that tests value at the Boundaries of an input range

* Identify the maximum & minimum valid values
* Test values just below the minimum
* Test maximum & just above the range
* Test just below the maximum.

for an Example, if the permitted age is 18-60, test cases are 17, 18, 19, 50, 60, 61. BVA defects causes by incorrect Boundary conditions.

10) function defect & how can Be prevented:

A functional defect occurs when software does not perform a function according to the Requirements, ATM Scenario, The ATM withdrawal must follow transaction logic, available balance must be validated, Correct amount must be transacted Using denominations rules. for an Example withdrawal of ₹10,250, the denominations must be 10,000 in forms of 500 (or) 2000, for 250, 200, 50 combination. Tester Should report defects and retest corrected functionality.

11) Defect Clustering:

It's the Principle that most defects are often concentrated in a small no of software modules.

* Analyze defect reports from previous test cycles.
* Identify Modules Containing the highest no of defects
* Investigate each defect complexity, risk
* Allocate more testing resources to those modules

for an eg: if most defect occurs in payment module, prioritize payment calculation, refunds. This improves testing efficiency.

Data inconsistency between modules: It occurs when connected module store, transfer or display different value for same information

* Review interface and data mapping
* Check data transfer between modules
* Verify database updates & synchronization
* perform integration testing, report inconsistency.

In health care system, that person patient updated address may appear in one module, but not other integration testing helps.

23) Test coverage measures how much of software, requirements or testable functionality has been tested. It includes identifying all requirement & important functions, Mapping test cases to requirements, Test normal & invalid inputs, include boundary, integration and peak loaded scenarios. Review uncovered area, add test cases. Shopping Application should test product search, cart operations and others. Better Coverage reduces risk of failure.

24) A Defect report may state that the payment page crashes after Clicking 'Pay', Clear documentation helps developers to reproduce. Defect documentation records software defects systematically for tracking and resolution.

* Assign unique defect ID, Record defect title
* Include steps to reproduce the defects
* Record expected & Actual result, severity, priority.

* Update the status until defect is found & verified.

29) Verify That money is deducted from sender & credited the receiver exactly one. A Banking System requires Systematic, testing Because transactions errors can cause financial losses.

* Review Banking regs & transaction rules
* Test deposits, withdrawal & fund transfers.
* Validate account Balance & transaction limits
* Test Security, invalid inputs & failed transactions
* Perform regression testing after defect corrections.

30) Early defect detection:

finding an incorrect Salary solution during design or unit testing cost less that) correcting it after payroll, Early detection identifies problems before they progress to later development stages.

* Review Requirements at the beginning of the project
* perform Unit testing during development
* Analyze defects & correct them as early as possible.